
Algorithms

Exercise 04

will be presented on Monday, June 15th, 2026

Remarks

1. This exercise contains five possible exam tasks that cover Chapter 6 (*Dynamic Programming*).
2. In the exercise class, there won't be time to present solutions for all of them, but the tasks are all quite similar. (I have not decided on which tasks I will present. Either I will pick according to what I believe will be most helpful to you, or I will let you pick the tasks at the beginning of the exercise.)
3. Write in your answers directly on the work sheets.

Task 1: (Text Segmentation)

/ 11 P

Let $A[1..n]$ be a string.

You are given blackbox access to a function $\text{IsWord}(i, j)$, which computes whether $A[i..j]$ is a “word”.

We want to solve the **Text Segmentation** problem in its **decision version**.

Step 1: Formalization + recursive solution:

- (a) Describe in words the meaning of the values $\text{SPLITTABLE}(i)$ for an index i in the word.
(How did we define these values in the formalization step?)

Define: $\text{SPLITTABLE}(i) :=$

/ 1 P

- (b) Suppose we have computed the value $\text{SPLITTABLE}(i)$ for all indices i . How can we use this information to solve the *Text Segmentation Decision Problem* on $A[1..n]$?

Goal:

/ 1 P

- (c) Write down the recursive solution that we derived for $\text{SPLITTABLE}(i)$:

$\text{SPLITTABLE}(i) = \left\{ \begin{array}{l} \end{array} \right.$

/ 3 P

Exercise 04 Algorithms

Algorithm:

(d) The algorithm that was presented in the lecture is given an input string

$$A[1..6] = \begin{array}{|c|c|c|c|c|c|} \hline C & A & B & A & A & B \\ \hline 1 & 2 & 3 & 4 & 5 & 6 \\ \hline \end{array} \text{ of length } n = 6$$

and a word function $\text{IsWORD}(i, j)$ defined by the following table:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
$i = 1$	C False	CA False	CAB False	CABA False	CABAA False	CABAAB False
$i = 2$		A False	AB True	ABA True	ABAA False	ABAAB False
$i = 3$			B True	BA False	BAA False	BAAB False
$i = 4$				A False	AA False	AAB False
$i = 5$					A False	AB True
$i = 6$						B True

Execute the algorithm that was presented in the lecture by writing the computed values for $\text{SPLITTABLE}(i)$ into the SPLITTABLE array printed below:

$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$

/ 6 P

Task 2: (Edit Distance)

/ 11 P

The *dynamic programming*-algorithm $\text{EDITDISTANCE}(A[1..m], B[1..n])$ from the lecture computes the *edit distance* of two strings (the minimal number of necessary single-character insertions, deletions or substitutions needed to transform one string into the other).

Formalization & recursive solution:

- (a) Describe in words the meaning of the values $\text{EDIT}(i, j)$.
(Or, equivalently: How did we *define* these values during the formalisation of the problem?)

/ 1 P

- (b) After you have computed the values $\text{EDIT}(i, j)$ for all $i, j \in [k]$,
how do you get the edit distance for the two strings?

/ 1 P

Exercise 04 Algorithms

(c) State the *recursive solution* for $\text{EDIT}(i, j)$:

$\text{EDIT}(i, j) =$

/ 3 P

Algorithm:

(d) Run the EDITDISTANCE algorithm from the lecture on the input $A = \text{OWL}$ and $B = \text{WOOL}$.

Use the 2-dimensionale array $\text{EDIT}(i, j)$ that is printed below.

The circle markings indicate cells $\text{EDIT}(i, j)$ where $A[i] = B[j]$.

/ 6 P

	ϵ	W	O	O	L
	$j = 0$	$j = 1$	$j = 2$	$j = 3$	$j = 4$
ϵ $i = 0$					
O $i = 1$					
W $i = 2$					
L $i = 3$					

Task 3: (Longest Increasing Subsequence)

/ 11 P

For an array $A[1 \dots n]$ of integers,

we want to compute the **length** of a **longest increasing subsequence** of A .

In the lecture, we solved the problem with the Dynamic Programming algorithm $\text{LONGESTINCREASINGSUBSEQUENCE}(A[1 \dots n])$.

Step 1: Formalization + recursive solution:

- (a) Describe in words the meaning of the values $\text{LISBIGGER}(b, j)$ for two indices b and j in the array. (How did we define these values in the formalization step?)

Define: $\text{LISBIGGER}(b, j) :=$

/ 1 P

- (b) Suppose we have computed the value $\text{LISBIGGER}(b, j)$ for all indices b and j . How can we use this information to find the length of the longest increasing subsequence of $A[1 \dots n]$? (You may write “LISB” instead of “LISBIGGER” to save time.)

Goal:

/ 1 P

Exercise 04 Algorithms

(c) Write down the recursive solution that we derived for LISBIGGER(b, j):

(You may write “LISB” instead of “LISBIGGER” to save time.)

$$\text{LISBIGGER}(b, j) = \left\{ \begin{array}{|c|} \hline \\ \hline \end{array} \right. \quad / 3 \text{ P}$$

Algorithm:

(d) Execute the algorithm that was presented in the lecture on the input array

$$A[1..4] = \begin{array}{|c|c|c|c|} \hline 4 & 1 & 5 & 3 \\ \hline \end{array} \quad \text{of length } n = 4$$

1 2 3 4

by writing the computed values for LISBIGGER(b, j) into the 2-dimensional array printed below.
To help you, cells LISB(b, j) with $A[b] \geq A[j]$ have been marked with a circle.

/ 6 P

		$A[1..n] =$	$A[2..n] =$	$A[3..n] =$	$A[4..n] =$	$A[5..n] =$
		$\begin{array}{ c c c c } \hline 4 & 1 & 5 & 3 \\ \hline \end{array}$ $j = 1$	$\begin{array}{ c c c } \hline 1 & 5 & 3 \\ \hline \end{array}$ $j = 2$	$\begin{array}{ c c } \hline 5 & 3 \\ \hline \end{array}$ $j = 3$	$\begin{array}{ c } \hline 3 \\ \hline \end{array}$ $j = 4$	$\begin{array}{ c } \hline \\ \hline \end{array}$ $j = 5$
$A[0]$	$b = 0$					
$A[1] = 4$	$b = 1$		LISB(1, 2) 	LISB(1, 3) 	LISB(1, 4) 	LISB(1, 5)
$A[2] = 1$	$b = 2$			LISB(2, 3) 	LISB(2, 4) 	LISB(2, 5)
$A[3] = 5$	$b = 3$				LISB(3, 4) 	LISB(3, 5)
$A[4] = 3$	$b = 4$					LISB(4, 5)

Task 4: (Matrix Chain Multiplication)

/ 11 P

For a sequence $(M_1, \dots, M_k)$, we want to compute the **cost** of an **optimal multiplication order** for the matrix chain multiplication $M_1 M_2 \dots M_k$.

Like in the lecture, let $(r_{i-1} \times r_i)$ be the dimensions of matrix M_i , and let $(r_0, \dots, r_k)$ be the sequence of dimensions of the matrices.

In the lecture, we solved the problem with the Dynamic Programming algorithm `MATRIXCHAINMULTIPLICATIONCOST` $(r_0, \dots, r_k)$.

Step 1: Formalization + recursive solution:

- (a) Describe in words the meaning of the values $\text{COST}(i, j)$.
(How did we define these values in the formalization step?)

Define: $\text{COST}(i, j) :=$

/ 1 P

- (b) How did we reframe the goal of our algorithm in terms of these values $\text{COST}(i, j)$?

Goal:

/ 1 P

Exercise 04 Algorithms

- (c) Like in the lecture, assume that the cost of a single matrix multiplication of an $(r \times s)$ -matrix with an $(s \times t)$ -matrix is given by a function $c(r, s, t)$.

Write down the recursive solution that we derived for $\text{Cost}(i, j)$:

$$\text{Cost}(i, j) = \left\{ \begin{array}{l} \text{ } \end{array} \right.$$

/ 3 P

Algorithm:

- (d) Execute the algorithm that was presented in the lecture on the input $(r_0, \dots, r_4) = (3, 1, 2, 1, 2)$ by writing the computed values for $\text{Cost}(i, j)$ into the 2-dimensional array printed below. Assume $c(r, s, t) = r \cdot s \cdot t$ as the cost function for a single matrix multiplication.

				$r_4 = 2$	
			$r_3 = 1$	$\text{Cost}(4, 4) =$	$i = 4$
		$r_2 = 2$	$\text{Cost}(3, 3) =$	$\text{Cost}(3, 4) =$	$i = 3$
	$r_1 = 1$	$\text{Cost}(2, 2) =$	$\text{Cost}(2, 3) =$	$\text{Cost}(2, 4) =$	$i = 2$
$r_0 = 3$	$\text{Cost}(1, 1) =$	$\text{Cost}(1, 2) =$	$\text{Cost}(1, 3) =$	$\text{Cost}(1, 4) =$	$i = 1$
	$j = 1$	$j = 2$	$j = 3$	$j = 4$	

/ 6 P

Task 5: (Maximum Independent Set on Trees)

/ 11 P

Let T be a tree with root vertex r .

As in the lecture, we denote the subtree rooted in a vertex v by T_v , and the children of v by $N(v)$.

We want to compute the **size** of a **largest independent set** in T .

In the lecture, we solved the problem with the Dynamic Programming algorithm TREEMAXIS.

Step 1: Formalization + recursive solution:

- (a) Describe in words the meaning of the values $\text{MAXIS}(v)$ for a vertex v in the tree.
(How did we define these values in the formalization step?)

$\text{MAXIS}(v)$ denotes

/ 1 P

- (b) Suppose we have computed the value $\text{MAXIS}(v)$ for all vertices v .
How can we use this information to find the size of the largest independent set of T ?

/ 1 P

(c) Write down the recursive solution that we derived for $\text{MAXIS}(v)$:

$$\text{MAXIS}(v) = \left\{ \begin{array}{l} \end{array} \right.$$

/ 3 P

Algorithm:

(d) Execute the algorithm that was presented in the lecture on the tree given below by writing the computed values for $\text{MAXIS}(v)$ into the dotted circle at each vertex.

/ 6 P

